

Secure Art Gallery

Week 4 - Team and Requirements

BIBIFI Group 11 | Huseyin Simsek and Gustavo Pepe

Kent State CS 4/53401 - Secure Programming - Fall 2026

Purpose and current scope

The course project is a web application for an art gallery. This submission covers only Weeks 4-6: requirements, secure design, and a working database-backed authentication system. After login, a user can view seeded room information. An administrator-only page demonstrates server-side authorization.

The complete course system will eventually manage gallery movements. Event recording, occupancy, history and account management are not implemented in this milestone.

Repository and technology

GitHub: <https://github.com/hsimsek1/secure-art-gallery-group11>

Python 3.12, Flask, Flask-SQLAlchemy/SQLAlchemy, SQLite, Flask-Login, Werkzeug password hashing, Jinja and HTML/CSS. Flask-WTF supplies CSRF protection; python-dotenv loads local configuration. Pytest verifies the milestone. The application runs locally in a browser at 127.0.0.1.

Users and roles

Role	Current access
User/Guest	Login, view the room list, and logout.
Employee	Login, view the room list, and logout. The role is stored separately from Guest for later course work.
Administrator	The same access plus the protected administrator page. No account management interface is included.

Functional requirements

ID	Requirement
FR-1	Create and initialize a persistent SQLite database with Users, Persons, Rooms and a small authentication log.
FR-2	Seed one account for each role and fictional persons and rooms. Repeating initialization or seeding must preserve existing records.
FR-3	Authenticate a user with a username and password; reject incorrect credentials.
FR-4	Maintain a signed login session and allow the user to log out.
FR-5	Require login before displaying seeded rooms from SQLite.
FR-6	Allow only an Administrator to access the administrator demonstration page.

Initial security requirements

These requirements describe the controls retained in the Week 6 implementation. The table does not claim a security-complete final application.

ID	Requirement and implementation
SEC-1	Store salted password hashes using Werkzeug script. Never store plaintext passwords in the database.
SEC-2	Verify credentials before login. Use a generic invalid-credentials message.
SEC-3	Enforce login and administrator permissions in Flask routes; do not rely on hiding browser links.
SEC-4	Use Flask-Login with Flask signed sessions, HttpOnly and SameSite=Lax cookies, and a 30-minute session lifetime. Clear the session at login and logout.
SEC-5	Validate login input lengths and username characters on the server. Use CSRF tokens on login and logout forms.
SEC-6	Use SQLAlchemy bound queries for browser input. Let Jinja escape displayed text.
SEC-7	Generate local secrets and seed passwords; exclude .env, credentials, databases and the virtual environment from Git. Run locally with debugging off.
SEC-8	Log login success, login failure and logout with time and a user ID when known. Do not log passwords or tokens.
SEC-9	Limit request bodies to 16 KiB. This is a basic size check, not a comprehensive availability defense.

Basic security goals

Confidentiality: protect password material and show only authorized information.

Integrity: enforce roles on the server and use database constraints and safe queries.

Availability: provide a small, reproducible local application; reject oversized requests.

Accountability: retain a small authentication log. It is not a tamper-proof audit system.

Verification and boundaries

The reduced tests cover account login, invalid credentials, protected pages, logout, password hashes, role checks, database initialization and seeding, schema consistency, input validation and CSRF. The demonstration is Browser -> Flask -> SQLite.

This is a local classroom milestone. HTTPS deployment, broader attack defenses, full gallery behavior and comprehensive security testing are outside Weeks 4-6 and are not claimed complete. Persons are seeded reference records; there is no persons-management page. Users are login accounts and are separate from tracked persons.

Source

F26_BIBIFI_Project.pdf, pages 4-5: Week 4 setup, Week 5 secure design, and the detailed Week 6 implementation checklist. Pages 1-3 provide overall system and security context. This document follows the Fall 2026 schedule.